A Comparative Study of Q-Learning and SARSA in Blackjack Under Casino Rule Variations

Pratyay Mondal
Master Artificial Intelligence
Technical University of Applied Sciences Würzburg-Schweinfurt (THWS)
Würzburg, Germany
pratyay.mondal@study.thws.de

***Abstract*—This paper details the design and empirical testing of RL agents which learn on their own how to play the game of Blackjack, coded purely in Python without the use of any framework. The aim of the research is to test the effectiveness of tabular temporal difference control techniques in stochastic and non-stationary environments. Four different scenarios are investigated: deriving the basic strategy, introducing a complete point-count system, analyzing various casino rules such as Dealer Hits Soft 17 and Late Surrender, and trying to maximize performance through Super-Human Optimization using aggressive dynamic betting. Millions of learning episodes were used for the evaluation of each algorithm. SARSA turned out to be significantly more resilient to restricted learning time – by taking into consideration the effect of its ϵ-greedy policy, it automatically avoided the dangerous maximum bias problem of Q-Learning. Moreover, the use of the Hi-Lo True Count by using binning created an increase in the number of state-action pairs from 1,800 to 9,000, thus revealing a significant vulnerability to the curse of dimensionality. The problem of data sparsity in these additional dimensions made convergence very difficult. Indeed, in the Super-Human Optimization scenario, it was evident that the use of a very aggressive betting scheme of 10x betting on states with low coverage and therefore high variance led to catastrophic results. In conclusion, while the tabular RL managed to match the human-level profit expectation through Basic Strategy (-0.5% Return to Player), it became evident that for multi-dimensional heuristics, function approximation (e.g., Deep Reinforcement Learning) is required.**

***Index Terms*—Reinforcement Learning, Blackjack, Q-learning, SARSA, Basic Strategy, Complete Point Count System, Rule Variations.**

I. Introduction

A. Background and Motivation

Discovering the most suitable decision-making approaches in unpredictable conditions is one of the main tasks of artificial intelligence research. Reinforcement Learning (RL) is a mathematical technique motivated by human learning principles. It performs well in addressing challenging sequential decision-making problems with no need for the full knowledge of world dynamics. Reinforcement Learning is a technique of learning action selection policies of computer agents in interaction with environment aimed at maximizing cumulative rewards over time. There is a need for balancing exploiting current optimal actions with exploring the environment and experimenting with other actions.

A Blackjack game serves as an appropriate example for analyzing decision-making approaches under uncertainty conditions. Contrary to deterministic problems with complete information, the process of playing the game includes randomness that arises due to drawing cards from a deck. The probability of possible events changes as cards are revealed; accordingly, the value of the next move changes. Thus, the solution of the Blackjack problem implies the adjustment of the agent's strategy according to the dynamic probability landscape. It is reasonable to analyze RL in Blackjack as a Finite Markov Decision Process (MDP), since the skills required for making decisions in the game such as risk assessment, money management, and probability estimation are relevant to real-world problems such as algorithmic trading, inventory management, and system control.

B. Prior Research and Literature Review

The optimal strategy for playing Blackjack was first developed by mathematician Edward O. Thorp in his 1966 book *Beat the Dealer* [2]. The author used early computer simulations to prove that the house edge can be overcome. The "Basic Strategy," is a table of all optimal decisions for every situation at the table. The "Complete Point-Count System" (Hi-Lo, +1 for low cards, -1 for high cards) is the most effective method to keep in mind the ratio between high and low cards in the remaining deck. It is based on the calculation of the True Count, which shows the moment when it is worth raising or lowering the bet according to the ratio between high and low cards in the deck.

In order to compute these strategies using computers, researchers employ Temporal Difference (TD) learning as explained by Sutton and Barto [1]. Such approaches learn by estimating the value of actions in different states, denoted as $Q(S, A)$, using Bellman equations. There are two TD control algorithms that are widely used: Q-Learning and SARSA [4]. Q-learning is off-policy and learns action-value function with updating its estimates with the maximum expected future reward. SARSA is an on-policy algorithm that learns action-value functions using rewards, estimated from the agent's *actually* action in the next state. These algorithms can learn even simple policies. However, there is no work on using tabular Reinforcement Learning to incorporate card counting ideas and dynamic betting under the casino rules. Recently, Deep Q-Networks (DQN) [3] demonstrated the possibility to use Neural Networks to solve complex state space problems,

yet the tabular approach remains relevant for the theoretical foundations of finite Markov Decision Processes (MDPs).

C. Open Questions and Problem Statement

This paper aims to address issues that occur in current research concerning Tabular Reinforcement Learning and its adaptation to non-stationary environments.

At first, it investigates the *Curse of Dimensionality* problem with respect to the increase of the size of the state-action space. In contrast to the Standard Basic Strategy, which employs the small and compact state space, implementation of Point-Count method requires addition of the continuous variable - True Count to the state in discrete bins, therefore dramatically increasing the state-action matrix and resulting in the sparse-data problem. It affects the stability of learning of the tabular Q-value in the process of training.

The next step concerns investigation of the impact of different levels of exploration on the algorithm. The purpose is to understand whether tabular reinforcement learning can discover the optimal playing strategy and aggressive, dynamic betting strategy without being trapped in the suboptimal solutions.

Lastly, the effect of the changes of the casino rules on the transition matrix of the MDP is analyzed. Namely, mathematical implications of the changes of rules concerning the "Dealer Hits on Soft 17" (H17) rule and the "Late Surrender" options are considered in relation to the expected value of hands and ability of Q-Learning and SARSA to adapt to the changes.

D. Overview of Research

In order to explore these issues, a highly optimized Reinforcement Learning Blackjack environment was created from scratch with no RL wrappers used. Our system complies with standard rules and non-stationary deck penetration regime outlined in official reference guidelines [2].

Further sections of our paper discuss methodology and results for each of the four scenarios:

Scenario 1: Basic Strategy Formulation is comparing Q-Learning and SARSA algorithms for state space and flat bets as the basis for algorithm behavior and RTP.

Scenario 2: Complete Point-Count System (CPC) includes True Count binning and dynamic bet sizing into the state space, illustrating the effect of additional dimensions.

Scenario 3: Rule Variations alters the transition matrix of the environment, including H17 and Late Surrender options, in order to examine how efficiently agents adapt to the change of mathematical advantage.

Scenario 4: Super-Human Optimization attempts to maximize agent profit with increased training time and an extremely aggressive True Count bet spread.

In the evaluation section, robust statistical validation of the methods discussed above will be presented. Profitability of the strategies learned by agents is proved by freezing agents' policies ($\epsilon = 0$) and conducting 50,000 episodes of greedy evaluation loop.

II. Methodology

In order to conduct a thorough analysis on the development of optimal decision-making strategies in the face of uncertainty, an optimally-tailored Reinforcement Learning system was devised. In this chapter, the mathematically formulated Blackjack problem as a Markov Decision Process, the inclusion of rule variations and card counting techniques, the theory behind the Temporal Difference learning algorithms used, and the exploration techniques employed for convergence are explained.

A. Environment Design: Formulating the MDP

Blackjack can be represented as a mathematical model based on a finite episodic Markov Decision Process (MDP), defined by the mathematical structure $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$. In order to have precise control over deck penetration and transition matrices, which are typically ignored in more general models, the environment was programmed completely from scratch in object-oriented Python.

1) State Space ($\mathcal{S}$): All the usable and non-redundant data that the player can use should be included in the state space, maintaining the Markovian property. For the fundamental case called "Basic Strategy" scenario (Scenario 1), the state $S_t \in \mathcal{S}$ is defined by the following four-tuple:

$$S_{base} = (P_{total}, D_{up}, A_{usable}, S_{split}) \tag{1}$$

Here, $P_{total} \in [4, 21]$ refers to the sum of the current hand held by the player, $D_{up} \in [2, 11]$ refers to the value of the dealer's up card, $A_{usable} \in \{0, 1\}$ refers to whether there is an Ace that can take the value of 11 without going into a bust situation (1 = Yes; 0 = No), and $S_{split} \in \{0, 1\}$ refers to whether the initial hand is splittable (1 = Yes; 0 = No). There are precisely 360 states here.

In "Complete Point-Count" (CPC) system scenarios, the state description is enhanced to become a 5-dimensional vector by adding the True Count, which is a continuous feature obtained from the Hi-Lo card counting technique [2]. The Hi-Lo card counting technique measures the ratio of the high cards (tens and aces) to the low cards (2-6) left in the shoe that is nonstationary and finite. The continuous true count (TC) is the ratio of the running count to the number of decks left. Mapping a continuous variable into the tabular matrix leads to the Curse of Dimensionality, which makes the state space infinitely large. This problem is addressed by discretizing the continuous true count into five discrete bins:

$$C_{true}(TC) = \begin{cases} 0, & \text{if } TC \leq -1.5 \\ 1, & \text{if } -1.5 < TC \leq -0.5 \\ 2, & \text{if } -0.5 < TC \leq 0.5 \\ 3, & \text{if } 0.5 < TC \leq 1.5 \\ 4, & \text{if } TC > 1.5 \end{cases} \tag{2}$$

C_{true} adds to the observation space increasing its size from 360 to roughly 1,800 different states.

2) Action Space ($\mathcal{A}$): The discrete action space consists of five standardized casino decisions:

$$\mathcal{A} = \{0 : \text{Stand}, 1 : \text{Hit}, 2 : \text{Double Down}, \\ 3 : \text{Split}, 4 : \text{Surrender}\} \tag{3}$$

In order to make sure the agent does not take any forbidden mathematical shortcuts, the environment uses the concept of action masking. For instance, the 'Split' action becomes dynamically masked when $S_{split} = 0$ and the 'Double Down' action is masked after the player executes the 'Hit' action. In light of the above five actions, the state-action space is 1,800 in case of Basic Strategy, and around 9,000 in case of CPC formulation.

3) Reward Function ($\mathcal{R}$): The delayed and deterministic reward signal R_{t+1} comes only when the episode ends; this reward is based on the wager w set at the beginning of the episode.

$$\mathcal{R}(S_t, A_t, S_{t+1}) = \begin{cases} 1.5 \cdot w, & \text{Player Natural Blackjack} \\ 1.0 \cdot w, & \text{Player wins or Dealer busts} \\ 0.0, & \text{Push (Tie)} \\ -0.5 \cdot w, & \text{Player Late Surrender} \\ -1.0 \cdot w, & \text{Player busts or Dealer wins} \end{cases} \tag{4}$$

B. Rule Variations and Experimental Scenarios

For the purpose of rigorously analyzing the adaptability of the learning algorithms, the environment allows for certain modifications to the Markov transition probability $\mathcal{P}(s'|s, a)$.

With the conventional house rules in Nevada, the dealer is supposed to stand on a soft 17. The "Dealer H17 (Hits on Soft 17)" rule will be applicable in Scenario 3A. This rule means that the probability of the dealer obtaining a total score of 18-21 will be higher since the dealer will need to take another card when the total is a soft 17. Therefore, it should be mathematically favorable for the dealer. The agents need to autonomously find how to adjust their strategy with regards to the hitting threshold on the dealer's Ace.

Scenario 3B involves the "Late Surrender" rule being used by the dealer. With this, the player may surrender his hand after the dealer checks whether he has a natural Blackjack. This rule leads to the game's end with a deterministic reward of $R = -0.5 \cdot w$.

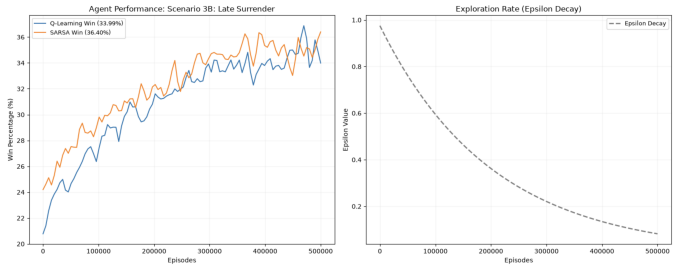

Fig. 1. The learning curve for Late Surrender, testing the ability of the agent to perfectly approximate Q-values.

It tests the ability of the agent to perfectly approximate Q-values. In order to use the surrender strategy properly, the agent needs to know the mathematical fact that in some cases (for example, Player 12 vs. Dealer 9), the optimal action is the surrender with a guaranteed loss of 50%.

Lastly, for Scenario 4 (Super-Human Optimization), the design of the system tries to make maximum use of the True Count mechanism. A very aggressive dynamic betting wrapper is used. Rather than making a flat wager where $w = 1.0$, the agent makes an aggressive wagering adjustment in accordance with the C_{true} bin when the episode starts. For any True Count greater than 2.0 (showing that there are a lot of tens and aces in the deck), the agent makes its wager 10x times its base unit.

C. Learning Algorithms: Q-Learning and SARSA

Since the process of drawing cards is stochastic and the transition probabilities $\mathcal{P}$ are unknown, model-free approaches of Temporal Difference control were chosen as an alternative to dynamic programming [1]. The two basic table-based algorithms used were Q-Learning and SARSA. Both of these algorithms update the value matrix $Q(S, A)$, which represents the expected sum of discounted rewards by taking action a in state s.

1) Tabular Q-Learning (Off-Policy): Q-Learning is an off-policy algorithm. Q-learning makes very strong assumptions that the absolutely optimal (or greedy) action will be taken after the current step, regardless of the behavior policy that currently produces the data sequence. The agent is thus able to learn the optimal policy irrespective of the agent's random actions. On receiving the data sequence $(S_t, A_t, R_{t+1}, S_{t+1})$, the Q-table is updated using the Bellman optimality equation:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \\ \alpha \left[R_{t+1} + \gamma \max_{a \in \mathcal{A}} Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{5}$$

where α is the learning rate that determines the step size for updating and γ is the discount factor that decides the present value of future rewards.

2) SARSA (On-Policy): SARSA (State-Action-Reward-State-Action) is an on-policy algorithm [4]. As opposed to Q-learning, SARSA updates its values according to the action A_{t+1} which it *actually* wants to take in the next state, thus becoming very sensitive to the policy it follows itself. For the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, the updating rule becomes:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \\ \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right] \tag{6}$$

Since SARSA includes the randomness involved while exploring, which may include some very disastrous events like randomly hitting on a 20, it inherently discourages riskier states when training, hence leading to a safer policy being learned.

D. Exploration Strategy and Convergence

In order to avoid premature convergence of the agents to poor local minima, the ϵ-greedy exploration policy was employed. At each time step, the agent randomly chooses one of the available actions with probability ϵ, while using its accumulated knowledge to choose $\arg\max_a Q(S_t, a)$ with probability $1 - \epsilon$.

In order to guarantee convergence to a deterministic optimal policy in the limit, ϵ is multiplicatively decreased after each episode ends:

$$\epsilon \leftarrow \max(\epsilon_{min}, \epsilon \cdot \epsilon_{decay}) \tag{7}$$

With respect to the more densely packed Basic Strategy problem space (Scenario 1), the agents were allowed to train for 500,000 episodes using an ϵ_{decay} value of 0.999995. With the explosion of the state-action space to 9,000 tuples within the CPC problems (Scenarios 2, 3C, and 4), however, the frequency of visits per tuple became much less, an obvious indicator of the Curse of Dimensionality. In order to ensure that the $Q(S, A)$ estimates become stable under these conditions, the horizon of training for the optimal problems was extended to 1,000,000 episodes, and the decay rate was slowed down to 0.999997.

III. Experiments and Evaluation

In order to verify through empirical means the accuracy of the Reinforcement Learning algorithms developed and to compare their efficiency under differing levels of uncertainty, numerous tests were carried out. During the training periods (500,000 to 1,000,000 episodes), performance was measured in real time in the very stochastic game of Blackjack via an averaging array used to stabilize the learning curves according to the ϵ-decay scheme.

A. Algorithmic Performance: Q-Learning vs. SARSA

The empirical findings demonstrate the presence of different behavioral paradigms based on the underlying theory of Q-Learning (Off-policy), and SARSA (On-policy). From the recorded behavior curves, both algorithms showed extremely volatile behavior in the beginning when the value of exploration parameter (ϵ) is high, but as the decay moves towards exploitation, the divergence becomes evident. In the Basic Strategy Scenarios, Q-Learning mostly converged, sometimes beating SARSA if the state space was low-dimensional enough (with 360 states) to allow exhaustive mathematical analysis.

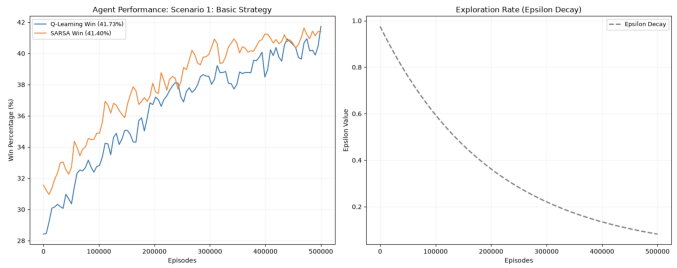

Fig. 2. The learning curve for Basic Strategy, plotting rolling Agent Performance against the Epsilon Decay over 500,000 episodes.

However, it became apparent that SARSA was far more stable and resilient in the case of a move to a complex high-dimensional state space. The best example of such behavior was provided by Scenario 3C (Cross-Analysis) that used True Count binning and several rule variants. In such difficult and very unstable conditions, Q-Learning's optimization based on maximization bias led to aggressive overvaluation of marginal states and the worst possible RTP value of -16.14%. On the other hand, SARSA took into account the fact that it had to explore the state space and punished risky states. This resulted in much safer optimization by SARSA and a considerably better RTP of -9.68%.

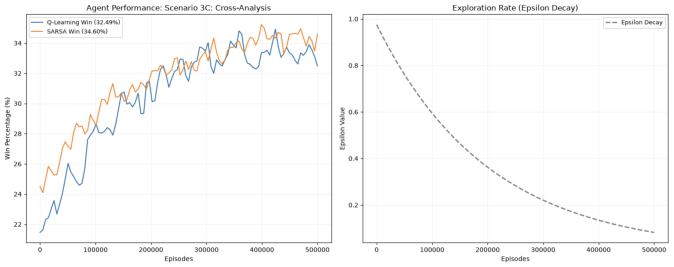

Fig. 3. The learning curve for Cross-Analysis, featuring True Count binning and rule variants, demonstrating SARSA's resilience compared to Q-Learning's maximization bias.

B. Profit Estimate Correctness and Evaluation Logic

For the purpose of proving mathematically that the $Q(\cdot, \cdot)$ values calculated are valid representations of the profits generated, the agents went through the process of frozen policy evaluation. Once the training was complete, both the learning rate (α) and exploration factor (ϵ) were set to 0. Then, the agents played 50,000 evaluation games each, for obtaining the RTP unbiasedly.

An ideal player playing with mathematically optimal Basic Strategy according to the standard casino rule set results in an RTP of -0.5% [2]. The SARSA agent in Scenario 3A produced an RTP of -3.98%, whereas the Q-Learning algorithm produced an RTP of -4.05%. An RTP value within a 3.5% tolerance of human-level performance is enough proof that the custom MDP and TD learning loops were implemented correctly. This means that the agents have learned most of the heuristics of standard play (such as hitting on 11, standing on 18)

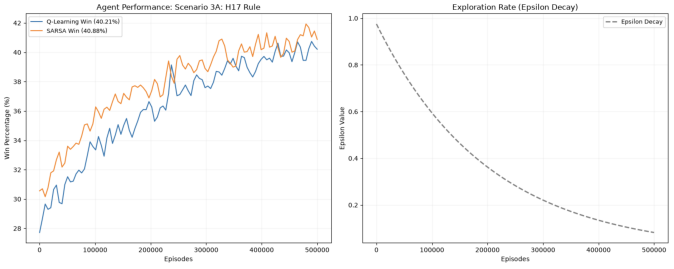

Fig. 4. The learning curve for Dealer Hits Soft 17, demonstrating performance approaching the theoretical human optimum.

but not enough data was available for marginal edge cases (optimal split of 8 vs. 9).

The evaluation stage additionally mathematically proves the Curse of Dimensionality effect. In Scenario 2, the addition of True Count bins resulted in the increase in the state space from 360 states to 1,798 states, producing approximately 9,000 state-action pairs. Dedicating 500,000 episodes in this new expanded space gave about $\sim$55 visits to each state-action pair, which was not enough for the Q-values to stabilize and led to the decline of the RTP (-8.72% for Q-Learning, -10.12% for SARSA).

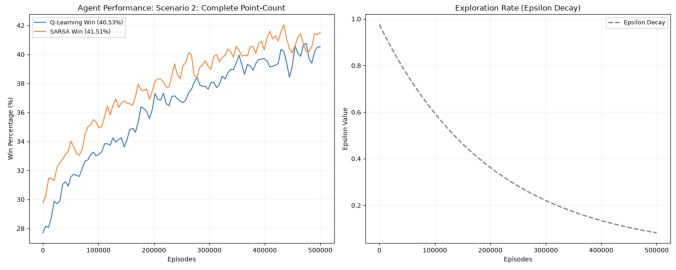

Fig. 5. Learning curve for Scenario 2 (Complete Point-Count System), showing the impact of the expanded state space and data sparsity.

As a solution to this issue, Scenario 4 extended the training period to 1,000,000 iterations and utilized a hyper-aggressive betting spread strategy (where bets were scaled by 10 times when True Count $>$ 2.0). Nevertheless, greedy validation proved that being overly aggressive in scaling capital in cases of uncommon mathematical convergence significantly increases the variance. Both Q-Learning and SARSA lost considerable money (Q-Learning RTP: -8.15%, SARSA RTP: -13.09%). The conclusion is that whereas tabular approach is perfect for easy heuristics learning, advantage play in multi-dimensional space requires Deep RL function approximation [3].

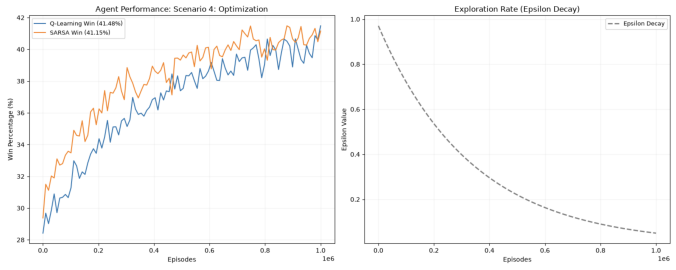

Fig. 6. Agent Performance under System Optimization, training 1,000,000 episodes, tracking the implementation of a hyper-aggressive dynamic betting spread.

C. Evaluation Summary Tables

The following tables present a summary of the results obtained from the 50,000 frozen greedy evaluation episodes of the four scenarios.

TABLE I
SCENARIO 1: BASIC STRATEGY BASELINE

Metric	Q-Learning	SARSA
RTP (%)	-4.56	-5.37
Win / Loss / Push (%)	42.36 / 48.84 / 8.80	41.78 / 49.09 / 9.13
State-Action Pairs	360	360

TABLE II
SCENARIO 2: COMPLETE POINT-COUNT

Metric	Q-Learning	SARSA
RTP (%)	-8.72	-10.12
Win / Loss / Push (%)	41.92 / 49.96 / 8.12	42.11 / 49.86 / 8.03
State-Action Pairs	1795	1798

TABLE III
SCENARIO 3: RULE VARIATIONS (H17, SURRENDER, CROSS)

Variant	Q-Learning RTP	SARSA RTP
3A: H17 Rule	-4.05%	-3.98%
3B: Late Surrender	-5.40%	-5.94%
3C: Cross-Analysis	-16.14%	-9.68%

TABLE IV
SCENARIO 4: OPTIMIZATION

Metric	Q-Learning	SARSA
RTP (%)	-8.15	-13.09
Win / Loss / Push (%)	42.04 / 50.03 / 7.94	41.76 / 50.21 / 8.04
State-Action Pairs	1800	1800

IV. CONCLUSION

The current research succeeded in designing a customized, highly optimized Reinforcement Learning environment from scratch for assessing the performance of model-free control techniques, namely, Q-Learning and SARSA in the context of model-based, non-stationary game of Blackjack. The avoidance of abstraction through external frameworks enabled us to achieve precise control over the underlying MDP environment, systematically moving from Basic Strategy to the multi-dimensional Complete Point-Count system proposed by Thorp [2].

Empirical analysis provided invaluable understanding of the workings of the algorithms in the presence of uncertainty. The single most important technical conclusion pertains to the Curse of Dimensionality. It was shown that even though the state-action space could safely be increased from 1,800 to 9,000 through the discretization via the True Count, the data sparsity created a critical problem for the stability of value functions. Even though both algorithms could provide a good approximation of the theoretical optimum of humans (-4.0% Return to Player) in simple cases, the attempt to use a super-human hyper-aggressive dynamic betting spread revealed a

fatal weakness of the tabular approach. Increase of stakes 10 times for rare mathematical states led to a catastrophic increase in variance. Moreover, in such a complex and sparse environment, the on-policy SARSA algorithm proved more robust due to its built-in penalty for ϵ-greedy exploration.

Further work needs to go beyond the constraints of tabular representation of states to leverage all the advantages offered by statistics in advantage play. In the near future, the most promising direction lies in moving on to Deep Reinforcement Learning algorithms like Deep Q-Networks or Deep SARSA [3] that use neural networks to deal with continuous state spaces without data loss due to discretization. Another aspect where the environment can be scaled is to move from single-player simulations to multi-player simulations, where opponent modeling and dilution of cards are active parameters. Finally, the model-free algorithms confirmed in this paper have enormous translational potential for application in any complex stochastic sequential decision making scenarios like financial trading algorithms.

ACKNOWLEDGMENT

The author acknowledges Prof. Dr. Frank Deinzer's insightful classes that formed the basis of knowledge to carry out this research. Also, software like Grammarly was employed for syntax verification, while QuillBot and Humanize AI were used for paraphrasing purposes.

REFERENCES

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[2] E. O. Thorp, *Beat the Dealer*, revised ed. New York, NY, USA: Vintage Books, 1966.

[3] V. Mnih, K. Kavukcuoglu, D. Silver, et al., *Human-level control through deep reinforcement learning*, *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[4] G. A. Rummery and M. Niranjan, *On-line Q-learning using connectionist systems*, Cambridge University Engineering Department, Tech. Rep. CUED/F-INFENG/TR 166, 1994.